

Kubernetes Platform Learning Guide

Date: 2026-08-17

Purpose

This guide explains what has been built so far in the kubernetes-platform project and how to study the code.

The project statement is:

I built a Kubernetes cluster foundation from scratch, then built a Kubernetes-native platform on top using CRDs and controllers.

The project is not just a collection of Kubernetes YAML. It now has a custom Kubernetes API, a controller that reconciles that API into runtime resources, a local kind development loop, in-cluster controller deployment, and HTTP routing through ingress-nginx.

Current Architecture

```
Developer
|
| creates App custom resource
v
Kubernetes API Server
|
| App event
v
App Controller running in-cluster
|
| reconciles desired state
v
Deployment -> Pods
Service -> stable cluster networking
Ingress -> host-based HTTP routing rule
ingress-nginx -> actual HTTP proxy
```

Local HTTP request path:

```
curl -H 'Host: demo.local' http://localhost:8080/
-> kind host port mapping
-> ingress-nginx controller
-> Ingress/demo-api
-> Service/demo-api
-> nginx Pod
```

Milestone 1: App CRD

What was built:

- A Kubernetes custom resource named App.
- The desired state API: image, port, replicas, and host.
- The observed state API: phase, URL, and conditions.

Why it matters:

Real platform teams often hide raw Kubernetes objects behind a simpler internal API. Instead of making users write Deployments, Services, and Ingresses directly, this project lets them declare one higher-level App.

Study these lines:

- platform/app-controller/api/v1alpha1/app_types.go:25-45

- AppSpec defines what the user wants.

- image is the container image.

- port is the app port.

- replicas is the requested scale.

- host enables HTTP routing.

- platform/app-controller/api/v1alpha1/app_types.go:47-61

- AppStatus defines what the controller observed.

- status belongs to the controller, not the user.

- platform/app-controller/api/v1alpha1/app_types.go:64-81

- The App object combines Kubernetes metadata, spec, and status.

Key concept:

spec = desired state
status = observed state

Interview question:

Why should status be updated by the controller instead of by the user?

Milestone 2: App Controller Reconciliation

What was built:

- A Go controller using controller-runtime.

- A reconcile loop that watches App resources.

- The controller creates or patches Deployments, Services, and Ingresses.

Why it matters:

Kubernetes is built around reconciliation. A controller watches desired state, compares it to actual state, and continuously repairs drift.

Study these lines:

- platform/app-controller/internal/controller/app_controller.go:60-69

- Fetches the App object.

- Handles the case where the resource was deleted.

- platform/app-controller/internal/controller/app_controller.go:71-90

- Builds and reconciles the Deployment.

- platform/app-controller/internal/controller/app_controller.go:92-112

- Builds and reconciles the Service.

- platform/app-controller/internal/controller/app_controller.go:114-134

- Builds and reconciles the Ingress when spec.host is present.

- platform/app-controller/internal/controller/app_controller.go:136-164

- Updates status phase, URL, and conditions.

Key concept:

CreateOrPatch makes the controller idempotent. Running reconcile once or many times should converge to the same desired state.

Interview question:

What happens if a user manually edits the generated Deployment?

Expected answer:

The controller should eventually reconcile it back to match the App desired state.

Milestone 3: Generated Runtime Resources

What was built:

- A generated Deployment for running app pods.
- A generated ClusterIP Service for stable in-cluster networking.
- A generated Ingress for HTTP routing.

Study these lines:

- platform/app-controller/internal/controller/app_controller.go:169-175
- Shared labels used for ownership, selectors, and lookup.
 - platform/app-controller/internal/controller/app_controller.go:177-182
- Default replica behavior.
 - platform/app-controller/internal/controller/app_controller.go:184-215
- Deployment generation.
 - platform/app-controller/internal/controller/app_controller.go:217-238
- Service generation.
 - platform/app-controller/internal/controller/app_controller.go:241-275
- Ingress generation.

Important detail:

platform/app-controller/internal/controller/app_controller.go:243 sets the Ingress class name to nginx.

That matters because ingress controllers may ignore classless Ingress objects depending on configuration. The platform should be explicit.

Interview question:

Why does the Service target port use the named container port http instead of only a number?

Milestone 4: Local kind Development Loop

What was built:

- A local kind cluster config.
- Scripts to create the cluster.

- Scripts to run the controller locally.
- Scripts to apply and validate the sample App.

Why it matters:

kind is not production Kubernetes, but it is useful for fast feedback. You can test controller behavior locally before deploying to a real kubeadm cluster.

Study these files:

- infra/kind/kind-cluster.yaml
- Maps host port 8080 to node port 80.
- Maps host port 8443 to node port 443 for a future HTTPS milestone.
- infra/kind/scripts/00-create-cluster.sh
- Creates the local kind cluster.
- infra/kind/scripts/02-run-controller-locally.sh
- Runs the controller from your laptop against the current kubeconfig.
- infra/kind/scripts/03-apply-sample-app.sh
- Applies the sample App.
- infra/kind/scripts/04-validate-app.sh
- Validates generated Kubernetes resources.

Key concept:

Running the controller locally is for development. Running the controller inside the cluster is closer to production.

Milestone 5: In-Cluster Controller Deployment

What was built:

- A local controller image build/load workflow.
- An in-cluster Deployment of the controller.
- Runtime validation that the controller works without make run.

Why it matters:

A real Kubernetes controller runs inside Kubernetes. It needs an image, Deployment, ServiceAccount, RBAC, leader election, and health checks.

Study these lines:

- infra/kind/scripts/05-build-load-controller.sh:4-7
- Computes the cluster name and image tag.
- infra/kind/scripts/05-build-load-controller.sh:17-23
- Checks Docker and kind prerequisites.
- infra/kind/scripts/05-build-load-controller.sh:31-33
- Builds the controller image and loads it into kind.

- infra/kind/scripts/06-deploy-controller.sh:4-8
- Computes the image tag and prepares to protect the tracked kustomization file.
 - infra/kind/scripts/06-deploy-controller.sh:10-13
- Restores Kubebuilder's kustomization.yaml after make deploy edits it.
 - infra/kind/scripts/06-deploy-controller.sh:35-38
- Deploys the controller and waits for rollout.

Important lesson:

The local image tag now includes the git SHA by default. This avoids a common kind problem: rebuilding the same image tag can leave the cluster running stale code.

Interview question:

Why can using the same local image tag cause confusing Kubernetes test failures?

Milestone 6: ingress-nginx HTTP Routing

What was built:

- ingress-nginx installation for kind.
- Explicit ingressClassName: nginx on generated Ingresses.
- End-to-end HTTP routing validation from localhost to the nginx sample app.

Why it matters:

An Ingress object is only desired state. It does not route traffic by itself. An ingress controller, such as ingress-nginx, watches Ingress resources and configures a proxy to implement those routing rules.

Study these lines:

- infra/kind/scripts/08-install-ingress-nginx.sh:4-6
- Pins the upstream ingress-nginx kind manifest version.
 - infra/kind/scripts/08-install-ingress-nginx.sh:16-23
- Checks Docker, kind, and kubectl.
 - infra/kind/scripts/08-install-ingress-nginx.sh:37-46
- Applies ingress-nginx, waits for readiness, and verifies the IngressClass and Service.
 - infra/kind/scripts/09-validate-http-routing.sh:4-9
- Defines the cluster, host, URL, repo root, and sample App.
 - infra/kind/scripts/09-validate-http-routing.sh:41-45
- Verifies the kind node exposes port 80 through host port 8080.
 - infra/kind/scripts/09-validate-http-routing.sh:47-54
- Waits for ingress-nginx and the app controller to be ready.
 - infra/kind/scripts/09-validate-http-routing.sh:56-58
- Applies the sample, forces a spec reconcile, then restores the sample state.

- infra/kind/scripts/09-validate-http-routing.sh:60-74

- Validates rollout, Service, Endpoints, Ingress host, and Ingress class.

- infra/kind/scripts/09-validate-http-routing.sh:76-85

- Sends a real HTTP request and checks for the nginx welcome page.

Key concept:

Ingress resource = routing rule stored in the Kubernetes API

Ingress controller = running proxy that implements the rule

Runtime proof completed:

```
curl -H 'Host: demo.local' http://localhost:8080/
```

returned the nginx sample app through ingress-nginx.

Interview question:

What is the difference between a Kubernetes Service and an Ingress?

Milestone 7: kubeadm Cluster Foundation

What was built:

- kubeadm bootstrap scripts.
- containerd setup.
- Kubernetes package setup.
- control-plane and worker join templates.
- validation and runbooks.

Why it matters:

Managed Kubernetes hides many operational details. kubeadm forces you to understand the control plane, kubelet, container runtime, CNI, certificates, and etcd.

Study these files:

- infra/kubeadm/README.md

- Main kubeadm walkthrough.

- infra/kubeadm/scripts/00-prereqs.sh

- Linux prerequisites.

- infra/kubeadm/scripts/01-containerd.sh

- container runtime setup.

- infra/kubeadm/scripts/02-kubernetes-packages.sh

- kubeadm, kubelet, and kubectl setup.

- infra/kubeadm/scripts/03-init-first-control-plane.sh

- first control-plane bootstrap.

- infra/kubeadm/scripts/06-validate-cluster.sh

- post-bootstrap validation.

- infra/kubeadm/runbooks/etcd-backup-restore.md

- operational recovery notes.

Key concept:

The kubeadm cluster is the production-style target. kind is the local development loop.

What You Should Be Able To Explain Now

You should be able to explain:

1. What a CRD is and why platforms use them.
2. The difference between spec and status.
3. What a reconcile loop does.
4. Why controllers use owner references.
5. How an App becomes a Deployment, Service, and Ingress.
6. Why a Service is needed even when Pods already have IPs.
7. Why an Ingress object does nothing without an ingress controller.
8. How kind maps local machine ports into the cluster.
9. Why local controller images need unique tags during testing.
10. Why kubeadm teaches different skills than managed Kubernetes.

Commands To Reproduce The Current Local Demo

Start Docker Desktop first.

```
cd /Users/sarige/kubernetes-platform
```

```
./infra/kind/scripts/00-create-cluster.sh
./infra/kind/scripts/05-build-load-controller.sh
./infra/kind/scripts/06-deploy-controller.sh
./infra/kind/scripts/08-install-ingress-nginx.sh
./infra/kind/scripts/09-validate-http-routing.sh
```

Useful inspection commands:

```
kubectl get app demo-api -o yaml
kubectl get deployment demo-api
kubectl get service demo-api
kubectl get ingress demo-api -o yaml
kubectl get endpoints demo-api
kubectl logs -n app-controller-system deployment/app-controller-controller-manager
kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller
curl -H 'Host: demo.local' http://localhost:8080/
```

Current Best Resume Story

Use this version:

Built a Kubernetes-native deployment platform by defining a custom App CRD and implementing a Go/controller-runtime reconciler that turns high-level app intent into Deployments, Services, and nginx-backed Ingress routing, validated end-to-end on kind with in-cluster controller deployment and HTTP traffic tests.

Shorter version:

Built a Kubernetes platform API with Go CRDs/controllers that reconciles apps into Deployments, Services, and Ingress routes, with local kind validation and ingress-nginx HTTP routing.

Next Milestone

The next milestone should be HTTPS with cert-manager.

Recommended scope:

- Install cert-manager.
- Add a local/self-signed or CA-backed certificate flow first.
- Update the App routing model to support TLS intentionally.
- Validate HTTPS separately from HTTP routing.

Do not combine this with GitHub Actions, ArgoCD, or observability yet. Keep the failure domains small.